

An infinite binary word avoiding Cartesian-tree squares of half-length at least four

Xiaojian Zeng

PBC School of Finance, Tsinghua University, 100190

xiaojianzeng@foxmail.com

September 8, 2026

Abstract

We construct an infinite word over $\{0 < 1\}$ in which no two adjacent blocks of the same length $p \geq 4$ have the same ordered Cartesian-tree shape, with ties resolved at the leftmost minimum. The four-uniform coding $h(a) = 0101$, $h(b) = 1111$, $h(c) = 0110$ works for every infinite square-free ternary word avoiding aba and cbc . Such a source is given explicitly by successive differences of binary digit-sum parity. A characterization of binary Cartesian-tree equivalence and a seven-bit synchronization table reduce all possible violations to half-lengths $4 \leq p \leq 12$. The remaining complete domain has 6,880 exact tree comparisons; a short verifier is included. This proves the binary threshold-four existence assertion listed as an experimental conjecture in the inspected version of Shibata, Mieno, Köppl, and Inenaga's *Relaxation of Square-Freeness*.

1 Statement and context

For a finite word s over an ordered alphabet, its Cartesian tree $\mathcal{T}(s)$ is the ordered, rooted, unlabeled binary tree defined as follows. The empty word gives the empty tree. Otherwise choose the *leftmost* occurrence of the minimum letter as the root; the subwords before and after it give the left and right subtrees recursively. Thus labels are discarded, while the distinction between left and right children is retained. Positions start at zero, and $s[i : j]$ denotes the factor with indices $i, \dots, j - 1$. A *Cartesian-tree square* is a factor uv with $|u| = |v| \geq 1$ and $\mathcal{T}(u) = \mathcal{T}(v)$. An ordinary square is a factor zz with z nonempty; a word is *square-free* if it has no ordinary square.

Shibata, Mieno, Köppl, and Inenaga [1] study square avoidance under several equivalence relations. Their Table 1 gives the binary Cartesian-tree maximum lengths 1, 5, 29, ? at thresholds 1, 2, 3, 4, respectively. Their Table 2 marks infinitude at threshold four as an experimental conjecture. Their Theorem 4.2 concerns order-preserving equivalence, a different relation. We prove the specified Cartesian-tree existence assertion using a uniform coding and a complete finite reduction. These comparisons refer to version 2 of the cited paper, inspected on September 8, 2026; they do not assert an exhaustive historical priority search.

Extend the following coding to finite and right-infinite words by concatenation:

$$h(a) = 0101, \quad h(b) = 1111, \quad h(c) = 0110. \quad (1)$$

Theorem 1. *Let y be any right-infinite square-free word over $\{a, b, c\}$ with no factor aba or cbc . Then $h(y)$ contains no Cartesian-tree square of half-length $p \geq 4$ over the ordered alphabet $\{0 < 1\}$.*

Corollary 2. *Let t_n be the parity of the number of ones in the binary expansion of n . Define $x_n = a, b, c$ according as $t_{n+1} - t_n$ is $+1, 0, -1$, and put $w = h(x)$. For every $i \geq 0$ and $p \geq 4$,*

$$\mathcal{T}(w[i : i + p]) \neq \mathcal{T}(w[i + p : i + 2p]).$$

In particular, an infinite binary word with the required avoidance exists.

We prove the coding theorem in Sections 2–5 and construct its source in Section 6. The proof combines a symbolic phase argument with exact finite checks. For adjacent methodological context, phase-based symbolic encodings also appear in work on reversible cellular automata [2]; that citation supplies no Cartesian-tree avoidance result or dependency of our proof.

2 The binary-tree invariant

A *descent* of a binary word s is an index $j \geq 1$ for which $s[j - 1 : j + 1] = 10$. Let $C(s)$ be the prefix ending at the zero in the last descent, or the empty word if there is no descent.

Lemma 3. *For binary words u, v of equal length,*

$$\mathcal{T}(u) = \mathcal{T}(v) \iff C(u) = C(v). \quad (2)$$

Consequently equal trees imply equality of all internal descent positions, as well as equality of the letters through the last descent.

Proof. If s has a zero, its root is the first zero, preceded only by a run of ones. The left subtree is therefore a right chain. Continuing along right children exposes successive zero-roots, each with the right chain of the preceding run of ones as its left subtree, and finally any terminal ones. A nonempty left subtree on this right spine marks precisely a zero at the end of a descent. If there is no such subtree, s is nondecreasing and its entire tree is a right chain; this includes words consisting solely of zeros or solely of ones.

Otherwise, the last spine vertex with a nonempty left subtree determines the last descent. Every spine vertex up to that point represents a zero, and the left-subtree sizes give all intervening runs of ones. The tree therefore determines $C(s)$ uniquely. The remaining subtree is a right chain with $|s| - |C(s)|$ vertices. Conversely, after this core, any nondecreasing suffix of that fixed length produces the same chain and hence the same tree. This proves (2), including the empty-word case. All descents occur within the core, proving the last assertion. $\square$

3 Seven-bit synchronization

Every three-letter factor of a source satisfying Theorem 1 belongs to

$$\mathcal{F}_3 = \{abc, aca, acb, bab, bac, bca, bcb, cab, cac, cba\}.$$

Indeed, square-freeness excludes equal adjacent letters, and the only further three-letter exclusions are aba and cbc . A seven-bit factor of $h(y)$ starting at residue r modulo four lies within three consecutive codewords, so it is $h(s)[r : r + 7]$ for some $s \in \mathcal{F}_3$. The four possible sets are displayed in Table 1.

Lemma 4. *Every nondecreasing factor of $h(y)$ has length at most six. Equal seven-bit factors of $h(y)$ start at the same residue modulo four.*

Proof. Every entry of Table 1 contains 10, and its four rows give pairwise disjoint sets. The table is obtained directly from (1) and the ten triples; the verifier in Appendix A also checks both properties. $\square$

r	Seven-bit factors starting at residue r
0	0101011, 0101111, 0110010, 0110111, 1111010, 1111011
1	1010110, 1011111, 1100101, 1101111, 1110101, 1110110
2	0101100, 0101101, 0111110, 1001010, 1001011, 1011110, 1101010, 1101011, 1101100, 1101101
3	0010101, 0010111, 0111101, 1010101, 1010111, 1011001, 1011011, 1111101

Table 1: The sets $\{h(s)[r : r + 7] : s \in \mathcal{F}_3\}$.

4 Excluding all large half-lengths

Suppose that uv is a Cartesian-tree square in $h(y)$ with $|u| = |v| = p \geq 13$. Lemma 3 gives a common core. Its omitted suffix in either half is nondecreasing, so Lemma 4 bounds the suffix length by six. Thus the halves agree on at least $p - 6 \geq 7$ initial bits. Synchronization forces $4 \mid p$; write $p = 4m$, where $m \geq 4$. Let $r \in \{0, 1, 2, 3\}$ be the starting residue of u .

Within each complete codeword the descent indicators at offsets 2 and 3 (with zero-based indexing) are

$$a : (1, 0), \quad b : (0, 0), \quad c : (0, 1).$$

These pairs distinguish the three source letters. Since all internal descent positions in u and v coincide, their corresponding complete codewords encode equal letters.

If $r = 0$, each half consists of m complete codewords, immediately giving two adjacent equal source blocks, a contradiction. If $r > 0$, the source factor covering the two halves has the form

$$\alpha U \beta V \gamma, \quad |U| = |V| = m - 1. \quad (3)$$

The halves begin with $(4 - r)$ -bit suffixes of $h(\alpha), h(\beta)$, contain $h(U), h(V)$, and end with r -bit prefixes of $h(\beta), h(\gamma)$, respectively. The complete codewords give $U = V$. The initial suffixes agree because they lie within the common prefix.

For $r = 1$, the three suffixes are 101, 111, 110; for $r = 2$, they are 01, 11, 10. Each list is injective. Hence $\alpha = \beta$ and (3) contains the ordinary square $(\alpha U)^2$.

It remains to treat $r = 3$. Equality of the initial bits means that the final bits of $h(\alpha)$ and $h(\beta)$ agree. If $\alpha = \beta$, the same square occurs. Otherwise $\{\alpha, \beta\} = \{a, b\}$, since only $h(c)$ ends in zero. If $\beta = \gamma$, the source contains $(U\beta)^2$, so assume also $\beta \neq \gamma$. The final three-bit prefixes are 010, 111, 011 for a, b, c , respectively; only the first has an internal descent. If $\beta = a$, equality of the descent indicators would force $\gamma = a$, contrary to our assumption. Thus $\alpha = a$ and $\beta = b$. The same comparison excludes $\gamma = a$, and $\gamma \neq b$, whence $\gamma = c$.

Let δ be the last letter of U , which exists because $m \geq 4$. It is also the last letter of V . At the boundary with the final partial codeword, $h(\beta)$ starts in one and $h(\gamma)$ starts in zero. If $h(\delta)$ ended in one, exactly one of these two boundaries would descend. Both boundaries lie inside their respective halves, contradicting Lemma 3. Hence $h(\delta)$ ends in zero, so $\delta = c = \gamma$. Now $V\gamma$ ends in the square cc . This contradiction exhausts all residues and excludes every $p \geq 13$.

5 The complete remaining finite domain

For $4 \leq p \leq 12$, a factor of length $2p$ starting at residue r meets at most

$$\left\lceil \frac{r + 2p}{4} \right\rceil \leq \left\lceil \frac{3 + 24}{4} \right\rceil = 7$$

source codewords. Its minimal covering source factor is square-free and avoids *aba* and *cbc*. We may therefore enumerate every admissible source word of lengths one through seven and compare the two trees at every fitting position and every half-length $4 \leq p \leq 12$ in its image. This covers every possible short violation, even though some enumerated words might not occur in a given infinite source.

The direct recursive-tree computation in Appendix A gives Table 2; every comparison is unequal. The code uses exact strings, integers, and tuples and is short enough to reproduce in full.

Source length	1	2	3	4	5	6	7
Admissible words	3	6	10	14	20	24	30
Tree comparisons	0	6	90	350	980	1944	3510

Table 2: The exhaustive finite domain: 6,880 comparisons, no violations.

The finite reduction and the large-half-length argument together prove Theorem 1. The restrictions on the source are substantive: the square-free finite word *cbc* has image 011011110110, containing 11011 11011 starting at bit one. Thus square-freeness alone is not a sufficient local coding criterion.

6 An explicit infinite source

The digit-parity word satisfies

$$t_0 = 0, \quad t_{2n} = t_n, \quad t_{2n+1} = 1 - t_n. \quad (4)$$

We give the needed square-freeness argument directly. An *overlap* of period p is a factor of length $2p + 1$ with $t_{i+j} = t_{i+p+j}$ for $0 \leq j \leq p$.

First, t has no overlap. Suppose one has period p and starts at i . If p is odd, each adjacent pair in the first $p + 1$ positions is unequal. When the pair starts at an even index this follows from (4); when it starts at an odd index, its equal copy shifted by p starts at an even index. The overlap equalities include both members of every such pair. The first $p + 1$ letters therefore alternate, so their endpoints differ because p is odd, contradicting $t_i = t_{i+p}$. If $p = 2q$ is even, take the letters at indices $i, i + 2, \dots, i + 4q$. By (4), these form a consecutive factor of t , or its bitwise complement, of length $2q + 1$ and period q . In either case t has an overlap of period q . Repeated halving reaches the impossible odd case. This proves the assertion for every $p \geq 1$.

Now set $d_n = t_{n+1} - t_n$, and rename $+1, 0, -1$ as a, b, c to obtain x . If x contained an ordinary square of half-length q at i , then $d_{i+j} = d_{i+q+j}$ for $0 \leq j < q$. Consequently

$$A_j = t_{i+q+j} - t_{i+j} \quad (0 \leq j \leq q)$$

would be constant. The values t_i, t_{i+q}, t_{i+2q} would form an arithmetic progression in $\{0, 1\}$, whose common difference must be zero. Thus all A_j vanish, yielding an overlap in t , a contradiction. So x is square-free. A factor *aba* in x would give three increments with sum 2, while a factor *cbc* would give sum -2 . Each such sum telescopes to the difference of two binary values, making either case impossible. Theorem 1 applies to this right-infinite x , and its nonerasing coding proves Corollary 2.

Computation and assistance

The proof-essential computation is exactly the finite domain in Table 2, together with the phase table. It was checked using two separately implemented tree representations and then recomputed in a separate model review context using parent vectors of inorder-numbered vertices. Additional checks covered 77,976 position–half-length pairs on admissible sources of lengths 8–12 and 211,584 pairs on lengths 13–16. These longer computations are finite consistency tests; the proof for all lengths is the reduction above. No floating-point computation is used.

OpenAI Codex assisted with proof development, exact verification code, manuscript preparation, and a separate-context mathematical review. That model review is not external peer review or statistically independent evidence. The article contains the mathematical argument and its critical finite verifier explicitly.

References

- [1] H. Shibata, T. Mieno, D. Köppl, and S. Inenaga, *Relaxation of Square-Freeness*, arXiv:2607.00674v2, revised July 14, 2026. <https://arxiv.org/abs/2607.00674v2>.
- [2] Zijian Zeng, *Exact Odd-Parameter Periods for Reversible Elementary Second-Order Cellular Automaton Rule 115*, 2026. <https://doi.org/10.2139/ssrn.7379379>.

A Executable finite verifier

The accompanying file `verify.py` is reproduced below in full. Run `python3 verify.py` without the `-O` option so that assertions remain enabled. It requires only the Python standard library. It prints phase sizes `[6,6,10,8]`, the two rows of Table 2, and a total of 6,880 comparisons.

```
"""Run with Python 3, without -O; no nonstandard packages are needed."""
from functools import lru_cache
from itertools import product

H = dict(a="0101", b="1111", c="0110")

def allowed(s):
    if "aba" in s or "cbc" in s:
        return False
    return all(s[i:i+p] != s[i+p:i+2*p]
               for p in range(1, len(s)//2 + 1)
               for i in range(len(s) - 2*p + 1))

def words(n):
    return [s for z in product("abc", repeat=n)
            for s in ["".join(z)] if allowed(s)]

def image(s):
    return "".join(H[c] for c in s)

@lru_cache(None)
def tree(s):
    if not s:
        return ()
    r = s.index(min(s))
    return (tree(s[:r]), tree(s[r+1:]))

phases = [{image(s)[r:r+7] for s in words(3)} for r in range(4)]
assert all("10" in z for phase in phases for z in phase)
assert all(phases[r].isdisjoint(phases[q])
            for r in range(4) for q in range(r))
print("phase sizes:", [len(s) for s in phases])
counts, total = [], 0
for n in range(1, 8):
    sources, tests = words(n), 0
    for s in sources:
        y = image(s)
        for p in range(4, min(12, len(y)//2) + 1):
            for i in range(len(y) - 2*p + 1):
                tests += 1
                assert tree(y[i:i+p]) != tree(y[i+p:i+2*p])
    counts.append(len(sources))
    total += tests
    print(n, len(sources), tests)
assert counts == [3, 6, 10, 14, 20, 24, 30]
assert total == 6880
print("total comparisons:", total)
```